

Национальный исследовательский ядерный университет «МИФИ»

Институт интеллектуальных кибернетических систем

Кафедра №12 «Компьютерные системы и технологии»



ОТЧЕТ

О выполнении лабораторной работы №4 «Работа со строками»

Студент: Сейтназаров Д.Н.

Группа: Б24-703

Преподаватель: Храпов А.С.

Москва – 2024

1. Формулировка индивидуального задания

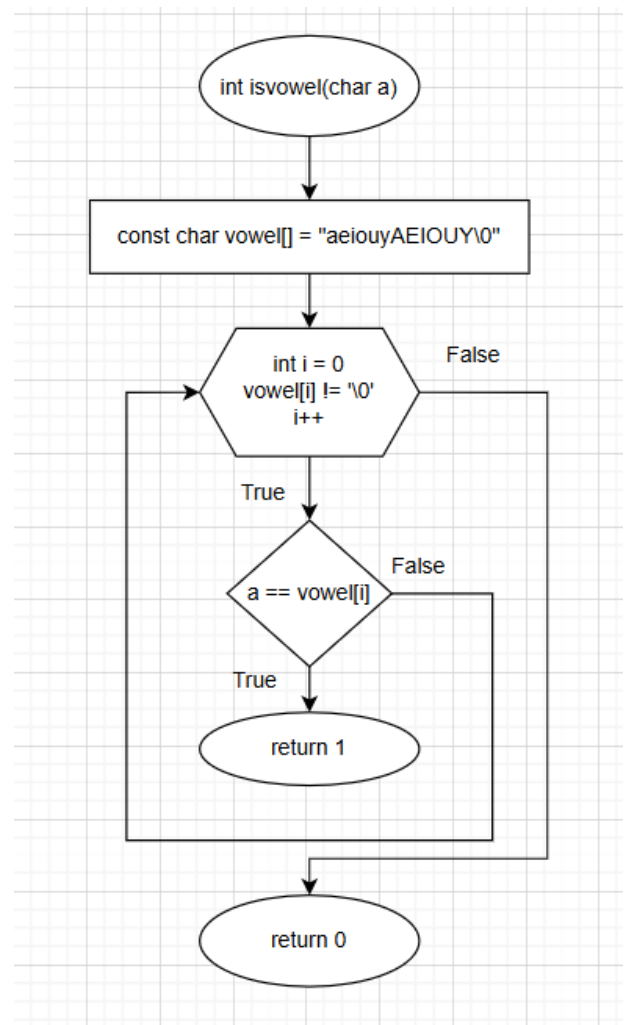
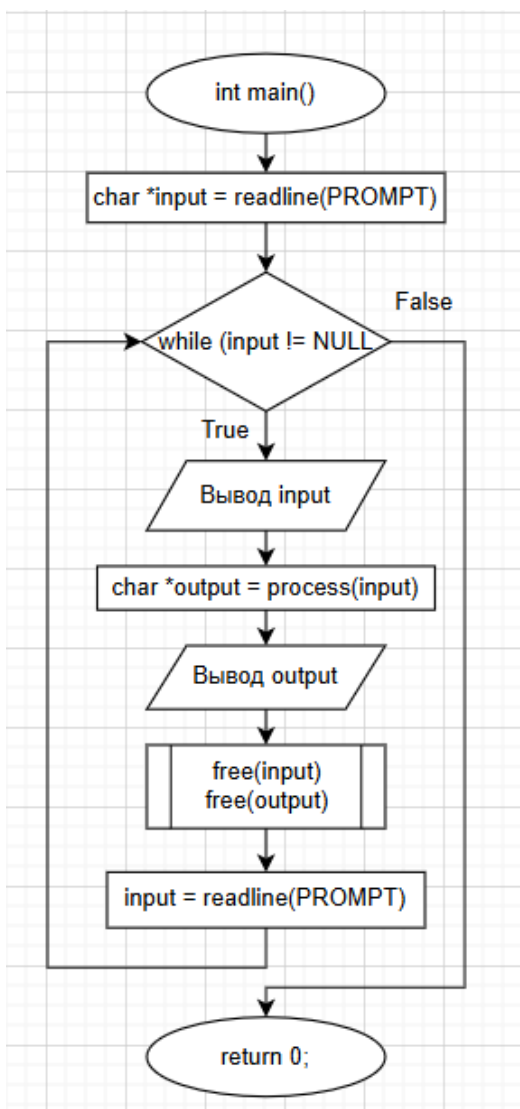
Вариант №14. Удалить из строки все слова с чётным количеством гласных.

2. Описание использованных типов данных

При выполнении данной лабораторной работы использовался встроенный тип данных `int`, `char`, `size_t`.

3. Описание использованного алгоритма

Рис. 1: Блок-схема алгоритма работ функций `main()`, `isvowel()`



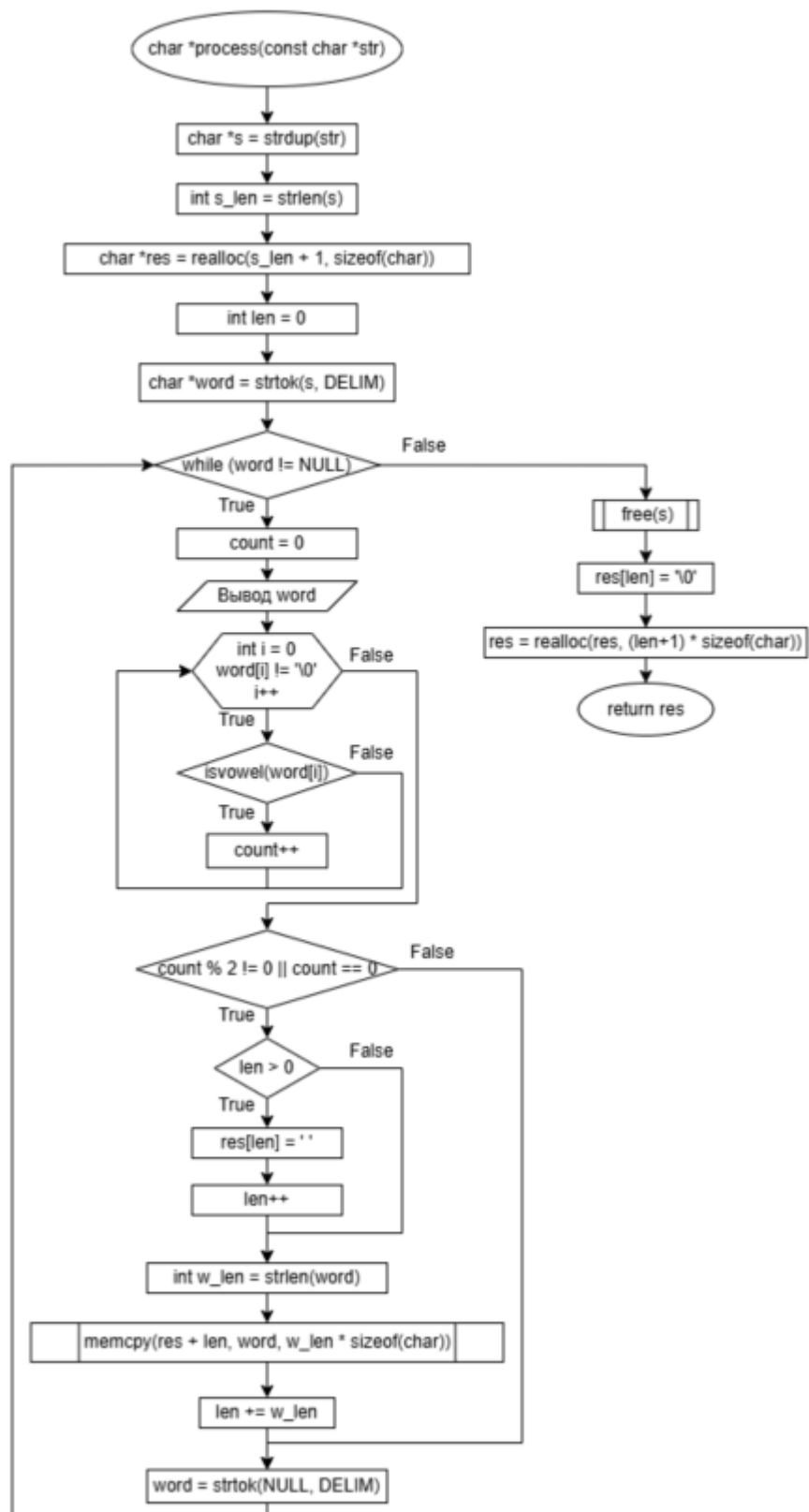


Рис. 2: Блок-схема алгоритма работы функции `*process()`

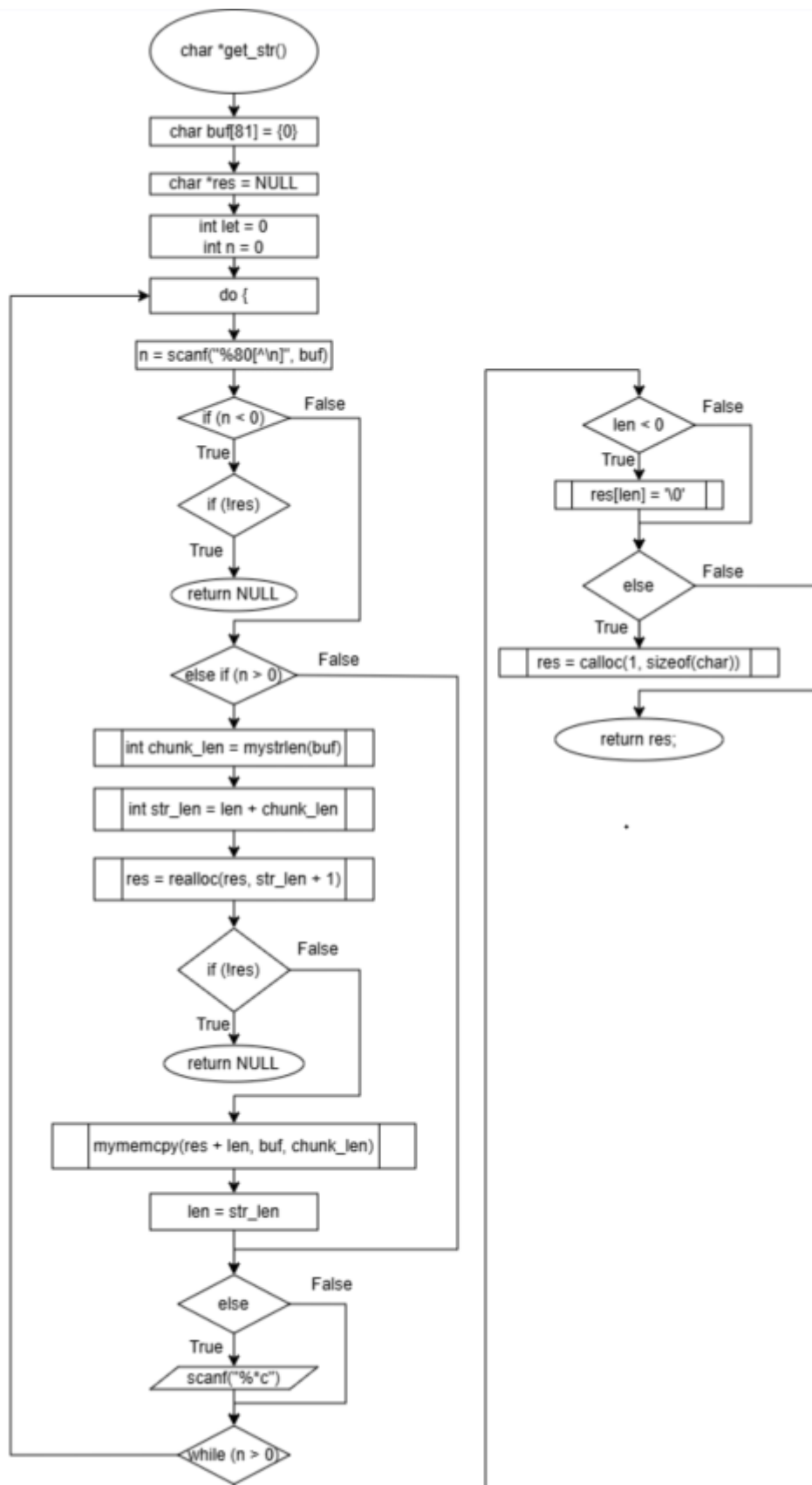


Рис. 3: Блок-схема алгоритма работы функции `*get_str()`

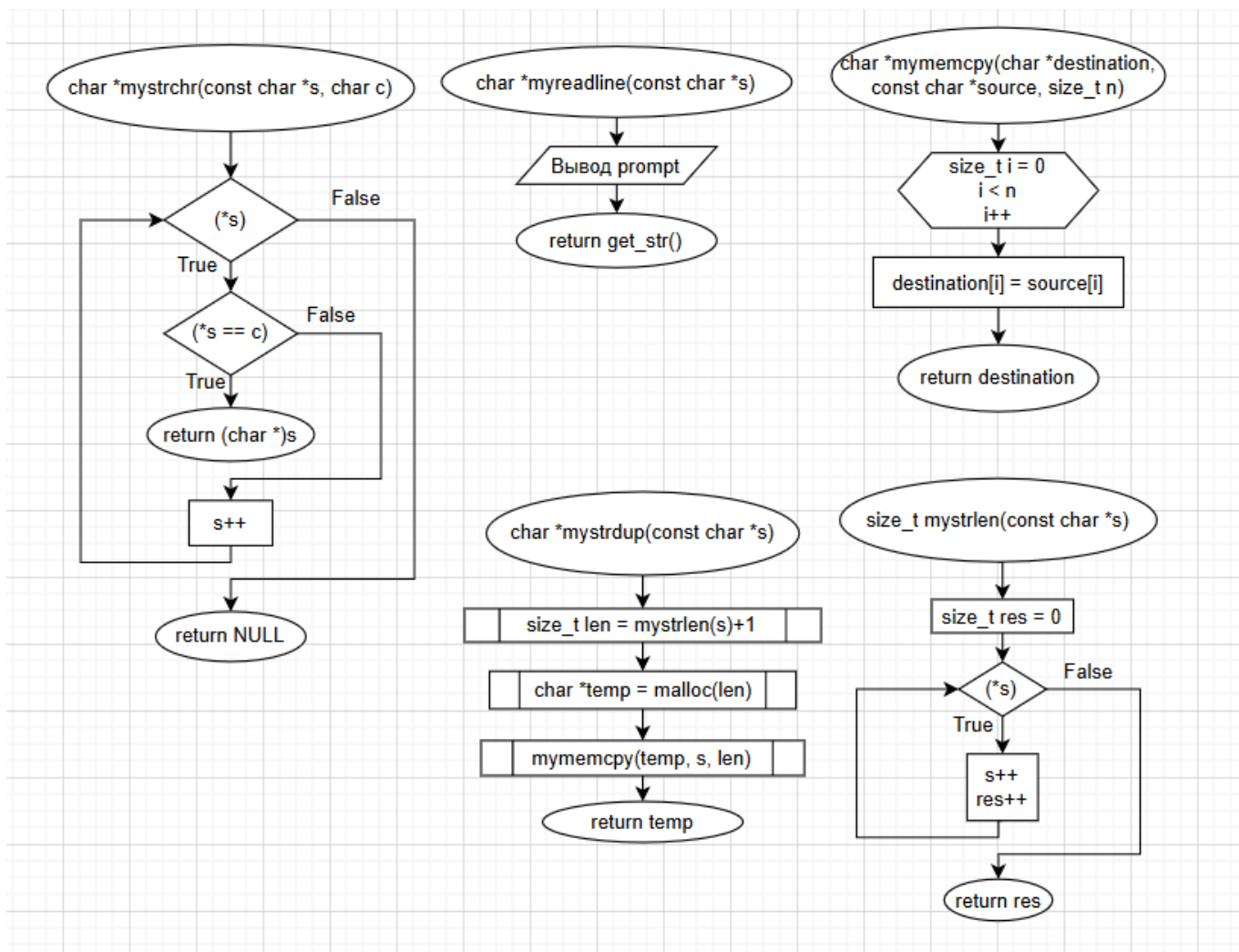


Рис. 4: Блок-схема алгоритма работ функций `*mystrchr()`, `myreadline()`, `memcpy()`, `mystrdup()`, `mystrlen()`

4. Исходные коды разработанных программ

Листинг 1: Исходные коды программы 4а (файл: lab4a.c)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <readline/readline.h>
4
5 #include <time.h>
6
7 #define DELIM " \t"
8 #define PROMPT "> "
9
10 char *process(const char *);
11 int isvowel(char a);
12
13 int isvowel(char a) {
14     const char vowel[] = "aeiouyAEIOUY\0";
15     for (int i = 0; vowel[i] != '\0'; i++) {
16         if (a == vowel[i]) {
17             return 1;
18         }
19     }
20     return 0;
21 }
22
23 int main() {
24     char *input = readline(PROMPT);
25     while (input != NULL) {
26         printf("%s\n", input);
27
28         clock_t start = clock();
29         char *output = process(input);
30         clock_t end = clock();
31
32         long double t = (long double)(end - start) / CLOCKS_PER_SEC;
33         if (output != NULL) {
34             printf("Result - %s\n", output);
35             printf("time - %Lf\n", t);
36             free(output);
37         }
38         free(input);
39         input = readline(PROMPT);
40     }
41     return 0;
42 }
43
44 char *process(const char *str) {
45     char *s = strdup(str);
46     if (s == NULL) {
47         return 0;
48     }
49     int s_len = strlen(s);
50     char *res = calloc(s_len + 1, sizeof(char));
51     if (res == NULL) {
52         free(s);
53         return 0;
54     }
```

```
55     int len = 0;
56     char *word = strtok(s, DELIM);
57
58     int count = 0;
59
60     while (word != NULL) {
61         count = 0;
62         printf("Word: \"%s\"\n", word);
63
64         for (int i = 0; word[i] != '\0'; i++){
65             if (isvowel(word[i]) == 1){
66                 count++;
67             }
68         }
69         if (count % 2 != 0 || count == 0) {
70             if (len > 0){
71                 res[len] = ' ';
72                 len++;
73             }
74             int w_len = strlen(word);
75             memcpy(res + len, word, w_len * sizeof(char));
76             len += w_len;
77         }
78         word = strtok(NULL, DELIM);
79     }
80
81     free(s);
82
83     res[len] = '\0';
84     res = realloc(res, (len + 1) * sizeof(char));
85     return res;
86 }
```

Листинг 2: Исходные коды программы 4б (файлы: main.c, mystring.c, mystring.h)

main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4 #include "mystring.h"
5 #define DELIM " \t" // Пробелы
6 #define PROMPT "> " // ?
7
8 int isvowel(char a) {
9     const char vowel[] = "aeiouyAEIOUY\0";
10    for (int i = 0; vowel[i] != '\0'; i++) {
11        if (a == vowel[i]) {
12            return 1; // Символ является гласной
13        }
14    }
15    return 0; // Символ не является гласной
16 }
17
18 int main() {
19     char *input = myreadline(PROMPT);
20     while (input != NULL) {
21         printf("%s\n", input);
22
23         clock_t start = clock();
24         char *output = process(input);
25         clock_t end = clock();
26
27         long double t = (long double)(end - start) / CLOCKS_PER_SEC;
28         if (output != NULL) { // Проверка на NULL
29             printf("Result - %s\n", output);
30             printf("time - %Lf\n", t);
31             free(output);
32         }
33         free(input);
34         input = myreadline(PROMPT);
35     }
36     return 0;
37 }
38
39 char *process(const char *str) {
40     char *s = mystrdup(str);
41     if (s == NULL) {
42         return 0;
43     }
44     int s_len = mystrlen(s);
45     char *res = calloc(s_len + 1, sizeof(char));
46     if (res == NULL) {
47         free(s);
48         return 0;
49     }
50     int len = 0;
51     char *word = mystrtok(s, DELIM);
52     int count = 0;
53 }
```



```

54     while (word != NULL) {
55         count = 0;
56         printf("Word: \"%s\\n\"", word);
57
58         for (int i = 0; word[i] != '\\0'; i++){
59             if (isvowel(word[i]) == 1){
60                 count++;
61             }
62         }
63         if (count % 2 != 0 || count == 0) {
64             if (len > 0){
65                 res[len] = ' ';
66                 len++;
67             }
68             int w_len = mystrlen(word);
69             memcpy(res + len, word, w_len * sizeof(char));
70             len += w_len;
71         }
72         word = mystrtok(NULL, DELIM);
73     }
74     free(s);
75     res[len] = '\\0';
76     res = realloc(res, (len + 1) * sizeof(char));
77     char *temp = realloc(res, (len+1) * sizeof(char));
78     if (temp == NULL){
79         free(res);
80         return NULL;
81     }
82     return temp;
83 }

```

mystring.c

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "mystring.h"
4
5 char *get_str() {
6     char buf[81] = {0};
7     char *res = NULL;
8     int len = 0;
9     int n = 0;
10
11     do {
12         n = scanf("%80[^\n]", buf);
13         if (n < 0) {
14             if (!res) {
15                 return NULL;
16             }
17         }
18         else if (n > 0) {
19             int chunk_len = mystrlen(buf);
20             int str_len = len + chunk_len;
21             res = realloc(res, str_len + 1);
22             if (!res) {
23                 return NULL;
24             }

```

```

25         memcpy(res + len, buf, chunk_len);
26         len = str_len;
27     }
28     else {
29         scanf("%*c");
30     }
31 } while (n > 0);
32
33 if (len > 0) {
34     res[len] = '\0';
35 }
36 else {
37     res = calloc(1, sizeof(char));
38 }
39 return res;
40 }
41
42 char *myreadline(const char *prompt) {
43     printf("%s", prompt);
44     return get_str();
45 }
46
47 size_t mystrlen(const char *s) {
48     size_t res = 0;
49     while (*s) {
50         s++;
51         res++;
52     }
53     return res;
54 }
55
56 char *mystrdup(const char *s) {
57     size_t len = mystrlen(s) + 1;
58     char *temp = malloc(len);
59     if (temp == NULL) {
60         return NULL;
61     }
62     memcpy(temp, s, len);
63     return temp;
64 }
65
66 char *memcpy(char *destination, const char *source, size_t n) {
67     for (size_t i = 0; i < n; i++) {
68         destination[i] = source[i];
69     }
70     return destination;
71 }
72
73 char *mystrtok(char *str, const char *delim) {
74     static char *token = NULL;
75     if (str != NULL) {
76         token = str;
77     }
78
79     if (token == NULL) {
80         return NULL;
81     }
82
83     char *first = token;

```

```

84     while (*first && mystrchr(delim, *first)) {
85         first++;
86     }
87
88     if (*first == '\0') {
89         token = NULL;
90         return NULL;
91     }
92
93     char *last = first;
94     while (*last && !mystrchr(delim, *last)) {
95         last++;
96     }
97
98     if (*last == '\0') {
99         token = NULL;
100     }
101     else {
102         *last = '\0';
103         token = last + 1;
104     }
105
106     return first;
107 }
108
109 char *mystrchr(const char *s, char c) {
110     while (*s) {
111         if (*s == c) {
112             return (char *)s;
113         }
114         s++;
115     }
116     return NULL;
117 }

```

mystring.h

```

1  #ifndef MYSTRING_H
2  #define MYSTRING_H
3  #include <stddef.h>
4
5  char *process(const char *);
6  int isvowel(char a);
7  char *get_str();
8  char *myreadline(const char *prompt);
9  size_t mystrlen(const char *s);
10 char *mystrdup(const char *s);
11 char *mymemcpy(char *destination, const char *source, size_t n);
12 char *mystrtok(char *str, const char *delim);
13 char *mystrchr(const char *str, char c);
14
15 #endif

```

5. Описание тестовых примеров

Таблица 1: 4a (lab4a.c)

input	output	time
happy new year	new year	0.018456

Таблица 2: 4a (lab4a.c)

input	output	time
love 12kaf	12kaf	0.000198

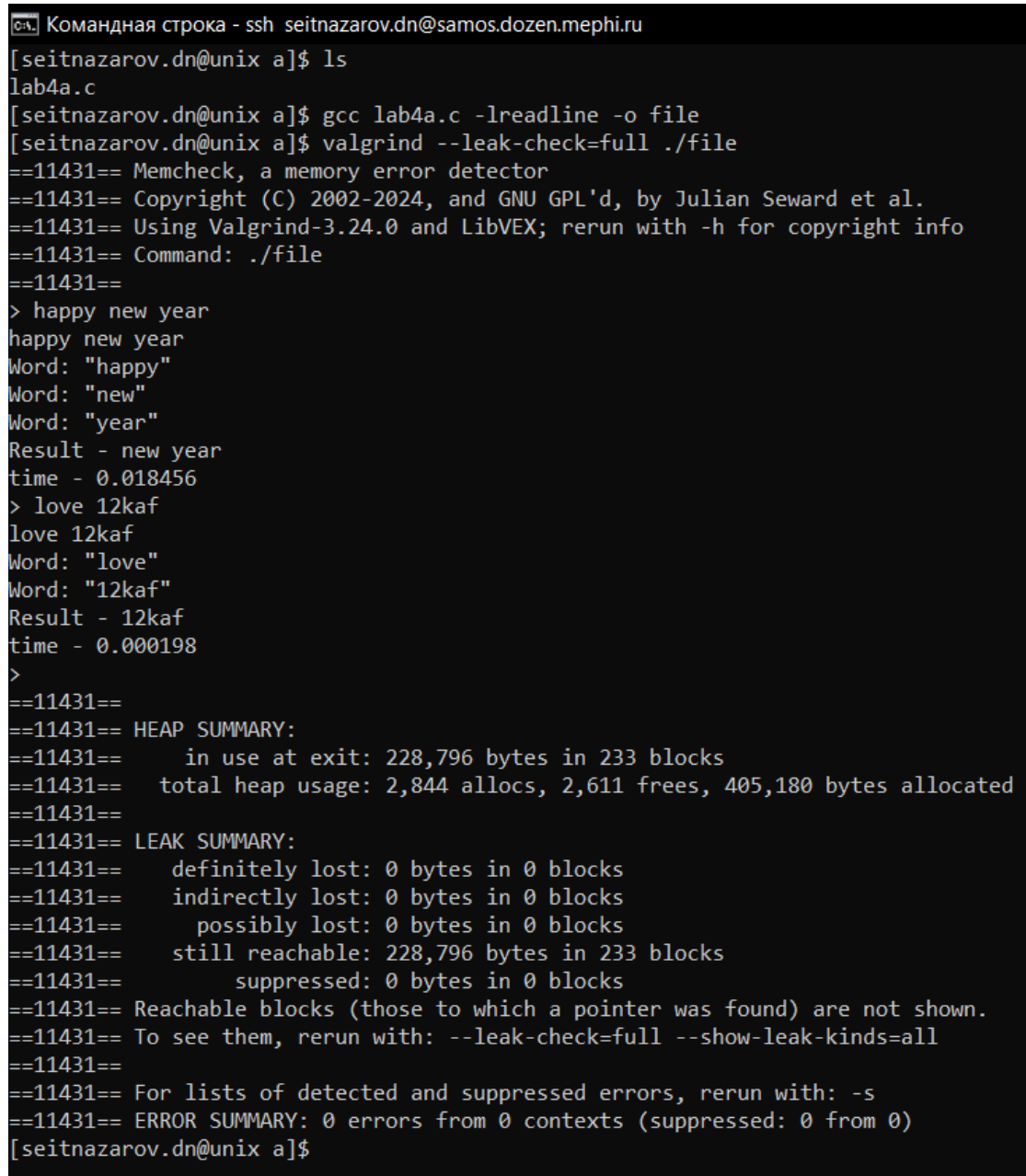
Таблица 3: 4б (main.c, mystring.c, mystring.h)

input	output	time
happy new year	new year	0.019521

Таблица 4: 4б (main.c, mystring.c, mystring.h)

input	output	time
love 12kaf	12kaf	0.000161

6. Скриншоты



```
Командная строка - ssh seitnazarov.dn@samos.dozen.mephi.ru
[seitnazarov.dn@unix a]$ ls
lab4a.c
[seitnazarov.dn@unix a]$ gcc lab4a.c -lreadline -o file
[seitnazarov.dn@unix a]$ valgrind --leak-check=full ./file
==11431== Memcheck, a memory error detector
==11431== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==11431== Using Valgrind-3.24.0 and LibVEX; rerun with -h for copyright info
==11431== Command: ./file
==11431==
> happy new year
happy new year
Word: "happy"
Word: "new"
Word: "year"
Result - new year
time - 0.018456
> love 12kaf
love 12kaf
Word: "love"
Word: "12kaf"
Result - 12kaf
time - 0.000198
>
==11431==
==11431== HEAP SUMMARY:
==11431==    in use at exit: 228,796 bytes in 233 blocks
==11431==   total heap usage: 2,844 allocs, 2,611 frees, 405,180 bytes allocated
==11431==
==11431== LEAK SUMMARY:
==11431==    definitely lost: 0 bytes in 0 blocks
==11431==    indirectly lost: 0 bytes in 0 blocks
==11431==    possibly lost: 0 bytes in 0 blocks
==11431==    still reachable: 228,796 bytes in 233 blocks
==11431==           suppressed: 0 bytes in 0 blocks
==11431== Reachable blocks (those to which a pointer was found) are not shown.
==11431== To see them, rerun with: --leak-check=full --show-leak-kinds=all
==11431==
==11431== For lists of detected and suppressed errors, rerun with: -s
==11431== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
[seitnazarov.dn@unix a]$
```

Рис. 5: Сборка и запуск программы lab4a.c

```
Командная строка - ssh seitnazarov.dn@samos.dozen.mephi.ru
[seitnazarov.dn@unix bprac]$ ls
main.c  mystring.c  mystring.h
[seitnazarov.dn@unix bprac]$ gcc main.c mystring.c -o file
[seitnazarov.dn@unix bprac]$ valgrind --leak-check=full ./file
==11245== Memcheck, a memory error detector
==11245== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==11245== Using Valgrind-3.24.0 and LibVEX; rerun with -h for copyright info
==11245== Command: ./file
==11245==
> happy new year
happy new year
Word: "happy"
Word: "new"
Word: "year"
Result - new year
time - 0.019521
> love 12kaf
love 12kaf
Word: "love"
Word: "12kaf"
Result - 12kaf
time - 0.000161
> ==11245==
==11245== HEAP SUMMARY:
==11245==      in use at exit: 0 bytes in 0 blocks
==11245==    total heap usage: 12 allocs, 12 frees, 2,156 bytes allocated
==11245==
==11245== All heap blocks were freed -- no leaks are possible
==11245==
==11245== For lists of detected and suppressed errors, rerun with: -s
==11245== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
[seitnazarov.dn@unix bprac]$
```

Рис. 6: Сборка и запуск программ main.c и mystring.c

7. Выводы

В ходе выполнения данной лабораторной работы, цель которой заключалась в том, чтобы удалить из строки слова с четным количеством гласных букв, я изучил:

1. Ввод/вывод строк.
2. Разработка функций для обработки строк.
3. Библиотека string.h.